

CRUX: HOW TO ALLOCATE AND MANAGE MEMORY

In UNIX/C programs, understanding how to allocate and manage memory is critical in building robust and reliable software. What interfaces are commonly used? What mistakes should be avoided?

Types of memory:

1. Stack:

↳ Allocations & deallocations are managed implicitly by the compiler.

↳ Also called automatic memory

```
void func() {  
    int x; // declares an integer on the stack  
    ...  
}
```

↳ The compiler makes space on the stack, when you call `func()` & deallocates it when you return from the function.

2. Heap:

↳ All allocations & deallocations are managed by the programmer.

```
void func() {  
    int *x = (int *) malloc(sizeof(int));  
    ...  
}
```

↓
lives on stack

↓
allocated on heap.

↳ Compiler makes room for `int *x` on stack.

↳ `malloc` allocates space on heap & returns the address which is stored on stack (in space allocated to `x`).

malloc() call

↳ Pass size to it. If successful it returns starting address of the space or else return NULL.

```
#include <stdlib.h>
```

```
...
```

```
void *malloc(size_t size);
```

no. of bytes
to be allocated

↳ Most commonly people do:

```
double *d = (double *) malloc(sizeof(double));
```

compile-time
operator.

↳ sizeof() substitutes with actual size (a number) at compile time.

↳ You can also pass variables to sizeof(), but in some cases you don't get desired results.

```
int *x = malloc(10 * sizeof(int));  
printf("%d\n", sizeof(x));
```

returns the size
of the address
(4 on 32-bit or
8 on 64-bit machines)

```
int x[10];  
printf("%d\n", sizeof(x));
```

In this case,
compiler has enough static information to know that 40 bytes have been allocated.

§ For strings use;

```
malloc(strlen(s) + 1)
```

§ malloc() returns a pointer to void type. This is because C doesn't know what type of data will be stored in the allocated space.

free() call

frees the memory allocated on heap.

```
int *x = malloc(10 * sizeof(int));
```

```
...
```

```
free(x);
```

§ Only the pointer returned by malloc is passed & not the size. Thus, size must be tracked by the memory allocation library.

Common Errors:

§ Many modern languages have garbage collector that frees space for you.

- Forgetting to allocate memory

§ Many routines expect memory to be allocated before you call them.

```
char *src = "hello";
```

```
char *dst; // oops! unallocated
```

```
strcpy(dst, src); // segfault and die
```

Correct code is:

```
char *src = "hello";  
char *dst = (char *) malloc(strlen(src) + 1);  
strcpy(dst, src); // work properly
```

- Not allocating enough memory

↳ A related error is NOT allocating enough memory, sometimes called buffer overflow

```
char *src = "hello";  
char *dst = (char *) malloc(strlen(src)); // too small!  
strcpy(dst, src); // work properly
```

↳ This will crash or work correctly because

- Overwriting a variable that isn't used anymore is harmless
- malloc() allocated a little extra space.

- Forgetting to Initialize Allocated Memory

- Forgetting to free memory

↳ Memory leak occurs when you forget to free memory.

In long running programs, this is a huge issue as slowly you run out of memory.

↳ No garbage collector will free a memory, if you still have reference to that memory.

- Freeing Memory Before You are Done with it

↳ Sometimes a program will free memory before it finished using it; such a mistake is called a dangling pointer.

↳ Subsequent use will crash the program or overwrite valid memory (call malloc again after free()).

- Freeing memory repeatedly

- ↳ Programs sometimes call `free()` more than once; this is known as double free.

- ↳ The result of doing is undefined as the memory-allocation library will get confused.

- Calling free incorrectly

- ↳ `free()` expects address returned from `malloc()`.

When you pass other value, bad things happen. This is known as Invalid free.

Underlying OS support

- ↳ `malloc()` & `free()` aren't system calls but rather library calls.

Thus, `malloc` library manages space in your virtual address space, but itself is built on top of system calls which call into the OS to ask for more memory or release some back to the system.

- ↳ One such system call is `brk`, which is used to change the location of the program's break: the location of the end of heap.

It takes one argument (address of the new break) & thus either increase or decrease the size of heap.

- ↳ `sbrk` is passed an increment (bytes to increase heap by) instead of address in `brk()`.

- ↳ You can also obtain memory from OS via `mmap()` call.

By passing correct args, `mmap()` can create an anonymous memory region within your program - a region associated with swap space.

↳ `calloc()` allocates memory & also zeroes it before returning.

↳ `realloc()` makes a new larger region of memory, copies the old region into it & returns the pointer to the new region.